# CAHIER DES CHARGES DEV COMPLET

## Cockpit Obsidia / X108

*Support narratif, fonctionnel et technique pour non-développeur et équipe dev*

Version 1.0 — générée le 2026-06-17

# Table des matières opérationnelle


- Partie 1 — Compréhension du projet
- Partie 2 — État existant
- Partie 3 — Architecture cible
- Partie 4 — Pages cockpit
- Partie 5 — Modèles de données
- Partie 6 — API Contract
- Partie 7 — Workflows
- Partie 8 — Sécurité
- Partie 9 — Backlog dev
- Partie 10 — Planning
- Partie 11 — Annexes
- Partie 12 — Guide non-développeur

# Cahier des charges DEV complet — Cockpit Obsidia / X108

Version 1.0 — Support de développement et de cadrage projet. Ce document est écrit pour un non-développeur, mais il doit aussi être exploitable par un développeur frontend, un développeur backend, un designer produit, un auditeur technique, un copilote IA et un futur associé. Il ne s'agit pas d'une note de vision. C'est un cahier des charges de construction. Il explique quoi construire, pourquoi le construire, comment l'organiser, quels écrans créer, quelles données afficher, quelles routes prévoir, quelles règles ne jamais violer et comment vérifier que le cockpit respecte la doctrine Obsidia / X108.

La règle de lecture est simple : le cockpit n'est pas un chatbot, pas un second kernel, pas un tableau décoratif et pas un raccourci de décision. Le cockpit est la tour de contrôle d'Obsidia. Il doit rendre visibles les couches déjà en place, leurs statuts, leurs preuves, leurs limites, leurs freezes, leurs mémoires, leurs branches, leurs agents, leurs routes, leurs logs et leurs validations. Il doit permettre de travailler avec le système sans l'abîmer.

Ce support utilise uniquement les éléments Obsidia / X108 présents dans le chat et les fichiers fournis. Quand une liste est confirmée, elle est reprise. Quand une liste est incomplète, elle est marquée LISTE PARTIELLE — SOURCE INCOMPLÈTE. Quand une capacité est souhaitée mais non prouvée comme déjà existante, elle est marquée À CRÉER ou NON CONFIRMÉ DANS LES SOURCES FOURNIES.

## À qui sert ce cahier des charges

- Pour Étienne : disposer d'un support maître qui explique tout le cockpit sans devoir réexpliquer Obsidia à chaque développeur.
- Pour un développeur frontend : comprendre toutes les pages à créer, les widgets, les états, les erreurs, les filtres et les interactions.
- Pour un développeur backend : comprendre les routes, registres, workers, schémas de données, jobs et services à créer sans toucher au kernel.
- Pour un designer UI : comprendre les intentions produit, les rôles utilisateurs et les priorités d'affichage.
- Pour un auditeur : voir les frontières d'autorité, les preuves, les do-not-claim, les freezes et les invariants de sécurité.
- Pour un copilote IA : savoir où se brancher pour assister sans décider.

## Ce que le document doit empêcher

- Empêcher de confondre cockpit et kernel.
- Empêcher de construire une interface jolie mais inutile pour l'audit.
- Empêcher de brancher Brody, Graphiti, Sigma ou un domaine comme autorité souveraine.
- Empêcher de perdre les preuves, les traces, les statuts, les memory candidates, les freezes et les décisions.
- Empêcher de vendre ou présenter le projet comme production-ready sans audit indépendant.
- Empêcher de démarrer par du frontend sans modèle de données ni workflows.

# Partie 1 — Compréhension du projet Obsidia / X108

## 1.1 Résumé narratif du projet

Obsidia / X108 est une architecture de gouvernance décisionnelle. Le projet ne cherche pas seulement à générer une réponse comme un chatbot, ni à automatiser une tâche comme un script classique. Il cherche à contrôler le passage entre une intention, une recommandation, une action possible et une action réellement autorisée. La question centrale n'est pas : que peut faire l'agent ? La question centrale est : qui a le droit d'autoriser l'action, à quel moment, avec quelle preuve, sous quelle trace, et avec quelle possibilité de replay ?

Dans un système agentique classique, un agent peut observer un contexte, raisonner, appeler un outil, écrire un fichier, envoyer une requête ou déclencher une action. Dans Obsidia, ces étapes sont séparées. Observer ne donne pas le droit d'agir. Mémoriser ne donne pas le droit d'autoriser. Expliquer ne donne pas le droit de décider. Afficher ne donne pas le droit d'exécuter. C'est le rôle du Kernel X108 : recevoir une action candidate, appliquer des invariants, produire un verdict ALLOW, HOLD ou BLOCK, et laisser une trace vérifiable.

Le cockpit est donc nécessaire parce qu'un tel système devient vite impossible à piloter avec un simple terminal ou une petite UI de chatbot. Il faut une interface qui montre l'état réel du projet : quels services sont actifs, quelles routes répondent, quelles décisions ont été émises, quelles mémoires sont candidates, quelles preuves sont passées, quels freezes existent, quels domaines sont branchés, quelles actions sont bloquées et quels éléments attendent une validation humaine.

## 1.2 Le vocabulaire minimal pour un non-développeur

• Kernel X108 — le noyau souverain. Il est le seul composant autorisé à produire un verdict final ALLOW, HOLD ou BLOCK.

• ALLOW — l'action candidate est autorisée par le kernel.

• HOLD — l'action est suspendue. Elle demande plus de preuve, plus de temps ou une validation humaine.

• BLOCK — l'action est interdite. C'est l'état le plus restrictif.

• Fail-closed — si un doute, une contradiction ou une panne existe, le système choisit HOLD ou BLOCK plutôt qu'un ALLOW permissif.

• FastAPI Bridge — l'API qui expose des routes au cockpit. Elle transmet et lit, mais elle ne décide pas.

• Sigma — la couche qui agrège les votes des agents de domaine et prépare un paquet pour le kernel.

• Brody — la couche d'éducation et d'assistance. Elle explique, apprend et propose, mais ne décide pas.

• Graphiti — la mémoire graphe en mode readonly, utilisée pour naviguer dans la connaissance déjà indexée.

• CIC — Causal Identity Context. C'est l'enveloppe causale d'un appel, avec 21 clés fixes pour prouver le rôle non souverain du composant.

• Proof Lab — l'espace qui exécute ou consulte les preuves Lean, TLA, Merkle, RFC3161 et les scripts de vérification.

• Freeze — un état figé, hashé et documenté. Une fois freeze, il sert de preuve d'état.

• Session Ledger — le journal d'une session : ce qui s'est passé, quel agent a agi, quelle décision a été prise, quelles preuves existent.

## 1.3 Les états réels confirmés par les sources

Cette section sépare les faits confirmés des intentions de build. Un développeur ne doit pas confondre ce qui existe déjà avec ce qui reste à créer dans le cockpit.

- FastAPI V5B sur 127.0.0.1:8000, 164 routes confirmées, rôle BRIDGE_ONLY, readonly dry-run.
- Kernel X108 Node sur 127.0.0.1:3001, fichier server.kernel.sealed.cjs, endpoint /kernel/ragnarok, confirmé par freeze mais fermé dans le dernier audit runtime.
- Graphiti V20 sur 8011, mode FROZEN_READONLY, snapshot 167 nœuds, 477 relations, 20 épisodes, dépendance Neo4j live inactive.
- Brody sur 8012, Education Pack V1 readonly, 124/124 tests PASS, two-level guard, PACK_READY_READONLY, PACK_INJECT_BLOCKED, P54_SANDBOX_NO_ACT.
- UI Vite sur 5173, surface existante mais fermée dans le dernier audit runtime.
- Neo4j 7474/7687, état dégradé ou fermé, AuthError à traiter avant live ingestion.
- CIC readonly context avec 21 clés fixes, authority=NONE, decision_authority=KX108_ONLY, flags d'action false.
- Sigma layer avec domaines BANK, TRADING, ECOM, GPS_DEFENSE_AVIATION, META et modules contracts, evaluate, registry, run_pipeline.

## 1.4 Ce que le cockpit doit faire comprendre dès la première page

Le premier écran du cockpit doit faire comprendre trois choses à un non-développeur. Premièrement, Obsidia est vivant : des services, mémoires, preuves, agents et domaines existent. Deuxièmement, Obsidia est contrôlé : les composants non souverains ne peuvent pas décider. Troisièmement, Obsidia est auditable : chaque décision, freeze, preuve, session et mémoire doit être traçable. Si le premier écran ne rend pas ces trois messages évidents, l'interface sera belle mais ratée.

# Partie 2 — État existant du système à respecter

## 2.1 Services, ports et rôles

### FastAPI — port 8000

Rôle : API bridge. Statut : CONFIRMÉ RUNTIME. Détail : Expose 164 routes, statut readonly, role BRIDGE_ONLY, decision_authority=KX108_ONLY.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

### Kernel X108 — port 3001

Rôle : Autorité souveraine. Statut : CONFIRMÉ FREEZE, fermé runtime. Détail : Fichier server.kernel.sealed.cjs, endpoint /kernel/ragnarok, seul composant qui décide.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

### Graphiti V20 — port 8011

Rôle : Mémoire graphe. Statut : CONFIRMÉ FREEZE, FROZEN_READONLY. Détail : 167 nœuds, 477 relations, 20 épisodes, pas de dépendance live Neo4j pour le snapshot.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

### Brody — port 8012

Rôle : Education / assistant readonly. Statut : CONFIRMÉ FREEZE. Détail : Education Pack V1, 124/124 tests PASS, two-level guard.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

### UI Vite — port 5173

Rôle : Interface existante. Statut : CONFIRMÉ FREEZE, fermé runtime. Détail : Surface UI existante à reprendre ou remplacer par le cockpit.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

### Neo4j HTTP — port 7474

Rôle : Base graph. Statut : DÉGRADÉ / fermé. Détail : AuthError ou service fermé, ne pas prétendre live ingestion.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

### Neo4j Bolt — port 7687

Rôle : Base graph. Statut : DÉGRADÉ / fermé. Détail : Même règle que Neo4j HTTP.

Implication cockpit : afficher ce service dans Runtime Center, indiquer son statut réel, éviter tout bouton qui lui donne une autorité qu'il n'a pas. Si le service est fermé, l'UI doit afficher l'état fermé au lieu de simuler une disponibilité.

## 2.2 Invariants existants à afficher partout

- decision_authority = KX108_ONLY
- api_role = BRIDGE_ONLY
- allowed_to_decide = false
- readonly = true
- advisory_only = true
- emits_act = false
- emits_verdict = false
- memory_write = false
- graphiti_write = false
- kernel_mutation = false
- neo4j_write = false
- mcp_bridge = false
- path_compute = false
- real_action = false
- canonical_write = false
- BLOCK > HOLD > ALLOW

Ces invariants ne sont pas de simples variables techniques. Ce sont les garde-fous du projet. Le cockpit doit les afficher dans les pages critiques, notamment Dashboard, Authority Graph, Security Center, Server Advisory, Domain Center, Memory Governance et Coding Sandbox. Un développeur doit comprendre qu'un bouton ou une route qui contredit ces invariants est un bug critique, pas une fonctionnalité.

## 2.3 Modules Sigma confirmés

Sigma est la couche d'agrégation. Elle contient des contrats, agents, registres, exemples et tests. Le cockpit ne doit pas réduire Sigma à un mot. Il doit permettre de voir ses modules, domaines, exemples et tests.

- sigma/aggregation.py
- sigma/base.py
- sigma/connectors.py
- sigma/contracts.py
- sigma/evaluate.py
- sigma/graphiti_readonly_bridge.py
- sigma/guard.py
- sigma/obsidia_sigma_v130.py
- sigma/orchestrator_preview.py
- sigma/packets.py
- sigma/protocols.py
- sigma/registry.py
- sigma/run_pipeline.py
- sigma/sigma_config.json
- sigma/sigma_monitor.py
- sigma/trees_activation_readonly.py
- sigma/domains/bank_agents.py
- sigma/domains/ecom_agents.py
- sigma/domains/gps_defense_aviation_agents.py

- sigma/domains/meta_agents.py
- sigma/domains/trading_agents.py
- sigma/stress_test_results.json

## 2.4 Connecteurs et domaines confirmés ou réservés

Les connecteurs ne sont pas des détails. Ils constituent le lien entre le monde métier et le système de gouvernance. Le cockpit doit les présenter domaine par domaine, avec statut, mode, tests, dernier run, dernier freeze et risque associé.

- connectors/bank_normal_flow.py
- sigma/domains/bank_agents.py
- connectors/trading_live.py
- sigma/domains/trading_agents.py
- sigma/examples/trading_domain_example_06.py
- connectors/aviation_robo.py
- sigma/domains/gps_defense_aviation_agents.py
- sigma/domains/ecom_agents.py
- connectors/context_packet_flow.py
- connectors/interface_ready_memory.py
- connectors/memory_feedback_candidate.py
- connectors/_decision_card_fr.py
- sigma/domains/meta_agents.py
- connectors/brody_memory_readonly_adapter.py
- NCP readonly stub – à conserver comme stub non souverain tant que le connecteur n'est pas implémenté
- Scraping readonly stub – aucun réseau externe, aucun fetch, aucun crawl
- Path Compute V0 advisory only – pas de décision, pas d'action, pas de mutation

## 2.5 Domaines à représenter

- Bank
- Trading
- GPS
- Aviation
- Defense
- Ecom
- CIC
- NCP
- Scraping readonly
- Meta
- futurs domaines identifiés dans les sources : ecommerce, defence, space, biology, healthcare

## 2.6 Proof stack à représenter

Le cockpit doit présenter les preuves comme une surface opérationnelle, pas comme une note obscure. Un auditeur doit pouvoir voir les scripts, rapports, statuts, tests passés, tests à relancer, preuves non exécutées et limites.

- Lean
- 57 théorèmes / lemmes
- lake build
- TLA
- TLC
- compute_merkle_root.py

- compute_merkle_standalone.py
- verify_all.py
- verify_decision.py
- verify_merkle.py
- Merkle
- RFC3161
- PROOFKIT_REPORT.json
- proofs/examples/README.md
- proofs/examples/manual_merkle_annotated.md
- proofs/examples/merkle_root_2201.md
- adversarial tests A1 – monotonie 1M paires
- adversarial tests A2 – fuzzing seuil
- adversarial tests B1 – collision Merkle 100K
- adversarial tests C1 – seal tamper
- adversarial tests D1 – consensus split fail-closed
- adversarial tests E1 – signature tamper
- smoke tests
- runtime probes
- OpenAPI snapshot
- MANIFEST_SHA256.json
- README_FREEZE.md
- audit reports
- replay reports
- decision traces
- hash reports
- freeze manifests
- rfc3161_anchor.json
- merkle_root.json
- metadata.json

## 2.7 CIC : les 21 clés à conserver

Le CIC est l'enveloppe causale. Chaque appel important doit pouvoir transporter ou référencer ces 21 clés. Dans le cockpit, elles doivent être lisibles dans les détails d'une session, d'un appel ou d'une décision.

- authority
- memory_write
- memory_type
- decision_authority
- api_role
- emit_act
- env
- epoch
- session_id
- ledger_hash
- pii_masking
- audit_mode
- freeze_after
- prover
- sigma_run_id
- brody_t

- terrain_branch
- proof_required
- user_intent
- component
- gate

# Partie 3 — Architecture cible du cockpit

## 3.1 Principe général

Le cockpit est construit comme une couche de lecture, d'orchestration légère, d'explication et de validation au-dessus du backend existant. Il ne doit pas refaire le kernel, ne doit pas refaire Sigma et ne doit pas inventer une mémoire indépendante. Il doit centraliser les états, les rendre lisibles et déclencher des demandes de validation ou de freeze quand nécessaire.

L'architecture cible se compose d'un frontend, d'un backend cockpit, d'un ensemble de registres, d'un collecteur d'événements, d'un système de stockage des sessions, d'un proof runner, d'un freeze manager et d'un module d'intégration avec les services existants. Chacun de ces éléments doit être explicite pour que le développeur sache quoi construire.

## 3.2 Frontend cockpit

- Framework recommandé : React + TypeScript + Vite ou Next.js selon le repository existant.
- Navigation principale : Dashboard, Authority, Runtime, Domains, Sigma, Memory, Knowledge, Brody, Validation, Proof, Resources, Ledger, Freeze, Sandbox, Server, Multimodal, Audit, Governance, Backlog.
- Composants UI requis : cartes de statut, tableaux filtrables, détails latéraux, timelines, graphes, diff viewer, log viewer, JSON viewer, markdown viewer, wizard de freeze, formulaire de validation, moteur de recherche global.
- Principe d'UX : chaque page doit expliquer son rôle en langage simple avant d'afficher les données techniques.
- Règle d'état vide : si une donnée est absente, afficher pourquoi elle manque au lieu d'afficher une page vide.
- Règle d'erreur : distinguer API_DOWN, KERNEL_DOWN, GRAPHITI_FROZEN, NEO4J_AUTH_ERROR, PROOF_NOT_RUN, LEGACY_SOURCE, DOCUMENTED_ONLY.

## 3.3 Backend cockpit

- Service à créer : apps/obsidia_api/routes/workcenter.py ou module équivalent si l'architecture actuelle impose une autre convention.
- Rôle : exposer des routes de lecture consolidées pour le cockpit.
- Règle : ne pas modifier le kernel, ne pas écrire dans Graphiti, ne pas produire de décision, ne pas contourner Sigma.
- Routes GET à créer : status, metrics, registries, services, routes, domains, sigma, memory, validation, brody, proofs, resources, sessions, freezes, sandbox, server, multimodal, audit, governance, backlog.
- Routes POST autorisées en V1 : créer une demande de validation, créer une memory candidate, clôturer une session dry-run, demander un freeze, demander un test process. Ces routes créent des demandes, elles ne réalisent pas l'action finale sans validation.

## 3.4 Registres à créer

- components_registry.json
- services_registry.json
- routes_registry.json
- domains_registry.json
- connectors_registry.json
- sigma_registry.json
- memory_registry.json
- knowledge_registry.json
- validation_registry.json
- training_registry.json

- proofs_registry.json
- resources_registry.json
- sessions_registry.json
- freezes_registry.json
- sandbox_registry.json
- server_registry.json
- multimodal_registry.json
- audit_registry.json
- governance_registry.json
- backlog_registry.json

Chaque registre contient une liste d'objets avec id, type, statut, vérité source, fichiers liés, routes liées, tests liés, freezes liés, risques, date de dernière vérification et actions autorisées. Ces registres servent à éviter la logique dispersée dans le frontend. Le cockpit lit les registres et affiche des vues cohérentes.

## 3.5 Stockage des événements

Le cockpit doit pouvoir reconstruire une session et un état projet à partir d'événements normalisés. Chaque événement doit porter project_id, domain_id, agent_id, session_id, trace_id, span_id, repo, branch, worktree, risk_level, knowledge_status, timestamp, payload et cic_context.

- session.created
- session.closed
- agent.selected
- prompt.sent
- prompt.received
- tool.called
- tool_call.started
- tool_call.finished
- command.started
- command.output
- file.read
- file.write
- file.patch_ready
- diff.generated
- test.executed
- test.passed
- test.failed
- policy.blocked
- policy.allowed
- human.validation_required
- human.approved
- human.rejected
- knowledge.ingested
- knowledge.classified
- knowledge.validated
- knowledge.canonized
- knowledge.published
- memory.updated
- proof.run.started
- proof.run.finished
- proof.passed
- proof.failed

- freeze.scheduled
- freeze.completed
- decision.emitted
- decision.replayed
- decision.confirmed
- external.alert
- error.occurred
- security.anomaly

# Partie 4 — Spécifications détaillées des pages du cockpit

## 4.1 Dashboard global

### Objectif de la page
Le Dashboard global est la porte d'entrée du cockpit. Il doit permettre de savoir en moins de trente secondes si Obsidia est stable, quelles couches sont vivantes, quelles couches sont fermées, quels éléments sont gelés, quelles preuves sont passées, quelles mémoires attendent une validation et quels risques sont ouverts.

### Utilisateur cible
Étienne, développeur, auditeur, futur associé, opérateur non technique. Cette page doit parler à tous sans obliger l'utilisateur à ouvrir un terminal.

### Explication non-développeur
Le Dashboard fonctionne comme le tableau de bord d'un avion. Il ne pilote pas l'avion, mais il indique si les moteurs répondent, si les alarmes sont allumées, si les instruments sont cohérents et si le pilote doit agir. Pour Obsidia, les moteurs sont FastAPI, Kernel X108, Sigma, Brody, Graphiti, CIC, Proof stack et les domaines. La page doit montrer la vérité réelle, pas une vérité optimiste.

### Données à afficher
- Statut FastAPI 8000 : LISTENING, CLOSED, ERROR, dernière sonde, nombre de routes.
- Statut Kernel 3001 : CONFIRMED_BY_FREEZE, CLOSED_RUNTIME, RUNNING_RUNTIME, dernière décision connue.
- Statut Graphiti 8011 : FROZEN_READONLY, nodes=167, relations=477, episodes=20, live_neo4j_dependency=false.
- Statut Brody 8012 : Education Pack, 124/124 tests PASS, PACK_READY_READONLY, PACK_INJECT_BLOCKED si guard.
- Statut UI Vite 5173.
- Statut Neo4j 7474 et 7687 : CLOSED, AUTH_ERROR, DEGRADED, RUNNING.
- Invariants actifs : decision_authority, api_role, readonly, emits_act, memory_write, graphiti_write, kernel_mutation.
- Décisions par domaine : Bank, Trading, GPS, Aviation, Defense, Ecom, CIC, NCP, Scraping readonly, Meta.
- Mémoires par statut : RAW_CAPTURE, SESSION_SCRATCH, CANDIDATE, NEEDS_HUMAN_REVIEW, NEEDS_PROCESS_REVIEW, PROCESS_VALIDATED, HUMAN_VALIDATED, PROMOTED_TO_CANON, PROMOTED_TO_TRAINING, PROMOTED_TO_OPERATIONAL, PROMOTED_TO_DOMAIN, PROMOTED_TO_PROOF, REJECTED, SUPERSEDED, DEPRECATED, FORBIDDEN_SECRET.
- Preuves : Lean, 57 théorèmes, lake build, TLA, TLC, verify_all.py, verify_decision.py, verify_merkle.py, Merkle, RFC3161, PROOFKIT_REPORT.json.
- Derniers freezes et derniers manifests.
- Tâches de validation humaine et process en attente.

### Widgets et composants UI
- Cartes de statut par service.
- Bandeau invariants de sécurité.
- Timeline des derniers événements.
- Compteur de décisions ALLOW, HOLD, BLOCK.
- Graphique de statut des proofs.
- Liste des alertes critiques.
- Bouton de navigation rapide vers chaque centre.

### Actions autorisées

- Rafraîchir les sondes.
- Ouvrir une page détail.
- Exporter un résumé dashboard.
- Créer une demande de validation à partir d'une alerte.
- Ouvrir le ledger associé à une alerte.

### Actions interdites

- Décider à la place du kernel.
- Relancer un service sans validation.
- Écrire dans Graphiti.
- Marquer un proof comme PASS sans test.
- Masquer une alerte critique.

### Routes backend à appeler ou à créer

- GET /api/workcenter/status
- GET /api/workcenter/metrics
- GET /api/workcenter/services
- GET /api/workcenter/proofs
- GET /api/workcenter/freezes
- GET /api/workcenter/validations
- GET /api/workcenter/memory/summary

### Modèles de données concernés

- ServiceStatus
- InvariantStatus
- DecisionSummary
- MemorySummary
- ProofSummary
- FreezeSummary
- ValidationQueueItem
- RiskAlert

### Tests à prévoir

- Tester chargement sans API : afficher API_DOWN.
- Tester kernel fermé : afficher CLOSED_RUNTIME sans générer de verdict.
- Tester Graphiti frozen : afficher FROZEN_READONLY.
- Tester aucun freeze : état vide explicatif.
- Tester mismatch route_count attendu / réel.

### Critères d'acceptation

- Toutes les couches principales sont visibles.
- Aucune donnée n'est inventée si le backend ne répond pas.
- Les invariants sont visibles en haut de page.
- Chaque alerte renvoie vers une page détail.
- Le dashboard se charge en moins de 5 secondes avec les données cached.

### Tickets dev minimum

- FE-DASH-001 Créer layout Dashboard.
- FE-DASH-002 Créer cartes services.
- FE-DASH-003 Créer bandeau invariants.
- BE-DASH-001 Créer endpoint status agrégé.

- BE-DASH-002 Créer endpoint metrics agrégé.
- QA-DASH-001 Tests états fermés et dégradés.

## 4.2 Authority Graph

### Objectif de la page

La page Authority Graph doit rendre visible la séparation d'autorité du projet. Elle prouve visuellement que seul Kernel X108 décide, pendant que FastAPI, Brody, Graphiti, Sigma, CIC, domaines et UI restent non souverains.

### Utilisateur cible

Non-développeur, designer, auditeur, développeur backend, personne externe qui doit comprendre en quelques minutes pourquoi Obsidia n'est pas un agent autonome classique.

### Explication non-développeur

Cette page est la carte constitutionnelle d'Obsidia. Elle montre qui a le droit de faire quoi. Dans un système classique, beaucoup de composants peuvent finir par agir indirectement. Dans Obsidia, l'autorité est rare. FastAPI transmet, Sigma agrège, Brody explique, Graphiti mémorise, CIC trace, le cockpit affiche, mais Kernel X108 seul décide.

### Données à afficher

- Nœud Kernel X108 : autorité KX108_ONLY, gate ALLOW/HOLD/BLOCK, endpoint /kernel/ragnarok.
- Nœud FastAPI : BRIDGE_ONLY, readonly, allowed_to_decide=false, emits_act=false, emits_verdict=false.
- Nœud Sigma : agrégation et contrats, DomainAggregate, X108Gate, non souverain.
- Nœud Brody : éducation, assistant, readonly, no act.
- Nœud Graphiti : FROZEN_READONLY, mémoire non souveraine.
- Nœud CIC : authority=NONE, 21 clés, receipts.
- Nœuds Domaines : Bank, Trading, GPS, Aviation, Defense, Ecom, CIC, NCP, Scraping readonly, Meta.
- Nœud Proof stack : vérifie, ne décide pas.
- Nœud Cockpit : observe, valide, demande, ne décide pas.

### Widgets et composants UI

- Graphe interactif avec couleurs par autorité.
- Panneau détail d'un nœud.
- Légende souverain / bridge / readonly / advisory / proof / frozen.
- Filtre par couche OS.
- Export PNG ou PDF du graphe.

### Actions autorisées

- Ouvrir détail d'un nœud.
- Afficher les invariants liés.
- Afficher les routes liées.
- Exporter le graphe.
- Créer un ticket si un composant a une autorité incohérente.

### Actions interdites

- Modifier l'autorité depuis l'UI.
- Créer un lien direct cockpit -> kernel.
- Déplacer Brody ou Graphiti en zone souveraine.
- Afficher un composant comme live s'il est seulement confirmé par freeze.

### Routes backend à appeler ou à créer

- GET /api/workcenter/authority/graph
- GET /api/workcenter/components
- GET /api/workcenter/routes
- GET /api/workcenter/invariants

### Modèles de données concernés

- AuthorityNode
- AuthorityEdge
- InvariantStatus
- ComponentCapability
- RouteBinding

### Tests à prévoir

- Vérifier que Kernel est le seul nœud souverain.
- Vérifier que Brody est non souverain.
- Vérifier que Graphiti est readonly.
- Vérifier que cockpit n'appelle pas directement kernel.

### Critères d'acceptation

- Un utilisateur externe comprend la hiérarchie en moins de 2 minutes.
- Aucun composant non kernel n'est affiché comme décideur.
- Les invariants apparaissent dans le panneau détail.
- Le graphe reste lisible à 20 nœuds minimum.

### Tickets dev minimum

- FE-AUTH-001 Créer graphe interactif.
- BE-AUTH-001 Créer registry authority nodes.
- BE-AUTH-002 Créer validation cohérence autorité.
- QA-AUTH-001 Test unicité nœud souverain.

## 4.3 Runtime Center

### Objectif de la page

Cette page doit surveiller les services, ports, healthchecks, logs et états runtime. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Runtime Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- ports 3001, 8000, 8011, 8012, 5173, 7474, 7687
- processus associés
- état LISTENING/CLOSED/DEGRADED/FROZEN_READONLY

- logs récents
- dernière sonde
- CPU, mémoire, uptime
- mode readonly
- erreurs API

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d’action contrôlés
- Zone risques et limites

### Actions autorisées

- Relancer une sonde
- Ouvrir les logs
- Créer une demande server advisory
- Exporter runtime_probe.txt

### Actions interdites

- Démarrer ou arrêter un service
- Tuer un processus
- Modifier env
- Afficher secrets

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-3-runtime-center/summary
- GET /api/workcenter/4-3-runtime-center/items
- GET /api/workcenter/4-3-runtime-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation

- La page explique son rôle sans jargon.

- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.3 -001 Créer page.
- FE-4.3 -002 Créer composants principaux.
- BE-4.3 -001 Créer endpoint summary.
- BE-4.3 -002 Créer endpoint details.
- QA-4.3 -001 Tests états et permissions.

## 4.4 Domain Center

### Objectif de la page

Cette page doit centraliser les domaines Bank, Trading, GPS, Aviation, Defense, Ecom, CIC, NCP, Scraping readonly, Meta et futurs domaines. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Domain Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- domain_id
- statut
- connecteur
- routes
- payload schemas
- dernier run
- dernier freeze
- dernier verdict
- risques
- preuves requises

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Voir détail domaine
- Simuler un payload dry-run
- Comparer deux domaines
- Créer demande ajout domaine

### Actions interdites

- Action réelle sans kernel
- domaine décide seul
- unverified live
- write sans validation

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-4-domain-center/summary
- GET /api/workcenter/4-4-domain-center/items
- GET /api/workcenter/4-4-domain-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.4 -001 Créer page.
- FE-4.4 -002 Créer composants principaux.
- BE-4.4 -001 Créer endpoint summary.
- BE-4.4 -002 Créer endpoint details.
- QA-4.4 -001 Tests états et permissions.

## 4.5 Sigma Center

### Objectif de la page
Cette page doit rendre compréhensible la couche Sigma, ses modules, agents, votes, contrats, agrégations et tests. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Sigma Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- modules Sigma
- agents par domaine
- DomainAggregate
- AgentVote
- X108Gate
- severity
- unknowns
- contradictions
- sigma_config.json

### Widgets et composants UI
- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées
- Afficher pipeline
- Lancer run_pipeline en simulation
- Lire tests Sigma
- Exporter aggregate

### Actions interdites
- Remplacer le kernel
- écrire une décision finale
- modifier sigma_config sans validation

### Routes backend à appeler ou à créer
- GET /api/workcenter/4-5-sigma-center/summary

- GET /api/workcenter/4-5-sigma-center/items
- GET /api/workcenter/4-5-sigma-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.5 -001 Créer page.
- FE-4.5 -002 Créer composants principaux.
- BE-4.5 -001 Créer endpoint summary.
- BE-4.5 -002 Créer endpoint details.
- QA-4.5 -001 Tests états et permissions.

## 4.6 Memory Governance Center

### Objectif de la page

Cette page doit gouverner toutes les mémoires, leurs statuts, promotions, validations et risques. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Memory Governance Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- SessionScratch
- Candidate
- Frozen
- KX108Ongoing
- Sigma
- BridgeCache
- Human Validated Memory
- Process Validated Memory
- Training Memory
- Operational Memory
- Domain Memory
- Proof Memory
- Canonical Memory
- Rejected Memory
- Deprecated Memory
- Forbidden Secret
- RAW_CAPTURE
- SESSION_SCRATCH
- CANDIDATE
- NEEDS_HUMAN_REVIEW
- NEEDS_PROCESS_REVIEW
- PROCESS_VALIDATED
- HUMAN_VALIDATED
- PROMOTED_TO_CANON
- PROMOTED_TO_TRAINING
- PROMOTED_TO_OPERATIONAL
- PROMOTED_TO_DOMAIN
- PROMOTED_TO_PROOF
- REJECTED
- SUPERSEDED
- DEPRECATED
- FORBIDDEN_SECRET
- ADVISORY
- PENDING_FREEZE
- FROZEN
- ARCHIVED

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d’action contrôlés
- Zone risques et limites

### Actions autorisées

- Voir mémoire
- Créer memory candidate

- Demander validation
- Demander freeze
- Rejeter candidate
- Promouvoir après validation

### Actions interdites
- Promouvoir sans humain
- écrire Graphiti
- supprimer frozen
- canoniser automatiquement

### Routes backend à appeler ou à créer
- GET /api/workcenter/4-6-memory-governance-center/summary
- GET /api/workcenter/4-6-memory-governance-center/items
- GET /api/workcenter/4-6-memory-governance-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés
- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.6 -001 Créer page.
- FE-4.6 -002 Créer composants principaux.
- BE-4.6 -001 Créer endpoint summary.
- BE-4.6 -002 Créer endpoint details.
- QA-4.6 -001 Tests états et permissions.

## 4.7 Knowledge Center

### Objectif de la page
Cette page doit gérer le cycle d'ingestion, classification, validation, canonisation et publication du savoir. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit

pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Knowledge Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- sources
- chunks
- classification
- liens Graphiti
- statut brut/classé/vérifié/canonique
- conflits
- preuves associées

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Ingest dry-run
- Classer
- Créer candidate
- Demander validation
- Lier ressource

### Actions interdites

- Publier canon sans validation
- utiliser source externe non tracée
- ingérer secret

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-7-knowledge-center/summary
- GET /api/workcenter/4-7-knowledge-center/items
- GET /api/workcenter/4-7-knowledge-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge

- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.7 -001 Créer page.
- FE-4.7 -002 Créer composants principaux.
- BE-4.7 -001 Créer endpoint summary.
- BE-4.7 -002 Créer endpoint details.
- QA-4.7 -001 Tests états et permissions.

## 4.8 Brody Education Center

### Objectif de la page
Cette page doit former l'utilisateur et Brody sur Obsidia sans donner d'autorité à Brody. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Brody Education Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- T1 — Comprendre Obsidia / X108
- T2 — Autorité et non-souveraineté
- T3 — Kernel X108
- T4 — API FastAPI readonly
- T5 — Bank domain
- T6 — Trading domain

- T7 — GPS / Aviation / Defense domain
- T8 — Brody education layer
- T9 — Graphiti readonly memory
- T10 — CIC / causal context
- T11 — Proof stack Lean / TLA / Python
- T12 — Freezes / manifests / audit reports
- T13 — Server launch and healthchecks
- T14 — Debugging without breaking kernel
- T15 — Patch proposal workflow
- T16 — Coding sandbox
- T17 — Replay / decision envelope
- T18 — Market / pitch / non-tech explanation
- T19 — Security boundaries
- T20 — Operational deployment readiness

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Lire module
- Répondre quiz
- Créer exercice
- Créer memory candidate
- Demander validation

### Actions interdites

- Brody décide
- Brody écrit mémoire
- Brody commit
- Brody contourne guard

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-8-brody-education-center/summary
- GET /api/workcenter/4-8-brody-education-center/items
- GET /api/workcenter/4-8-brody-education-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.8 -001 Créer page.
- FE-4.8 -002 Créer composants principaux.
- BE-4.8 -001 Créer endpoint summary.
- BE-4.8 -002 Créer endpoint details.
- QA-4.8 -001 Tests états et permissions.

## 4.9 Human Validation Center

### Objectif de la page
Cette page doit gérer les validations humaines nécessaires pour mémoire, freeze, patchs, domains, proofs, HOLD. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Human Validation Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- validation_id
- type
- objet ciblé
- raison
- risque
- assigné
- SLA
- statut
- commentaires
- preuve liée

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d’action contrôlés
- Zone risques et limites

### Actions autorisées

- Approve
- Reject
- Request test
- Request freeze
- Downgrade
- Promote

### Actions interdites

- Valider sans trace
- supprimer historique
- forcer PASS

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-9-human-validation-center/summary
- GET /api/workcenter/4-9-human-validation-center/items
- GET /api/workcenter/4-9-human-validation-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.9 -001 Créer page.
- FE-4.9 -002 Créer composants principaux.
- BE-4.9 -001 Créer endpoint summary.
- BE-4.9 -002 Créer endpoint details.
- QA-4.9 -001 Tests états et permissions.

## 4.10 Process Validation Center

### Objectif de la page

Cette page doit gérer les validations automatiques : tests, proofs, smokes, probes, manifests. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Process Validation Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- Lean
- 57 théorèmes / lemmes
- lake build
- TLA
- TLC
- compute_merkle_root.py
- compute_merkle_standalone.py
- verify_all.py
- verify_decision.py
- verify_merkle.py
- Merkle
- RFC3161
- PROOFKIT_REPORT.json
- proofs/examples/README.md
- proofs/examples/manual_merkle_annotated.md
- proofs/examples/merkle_root_2201.md
- adversarial tests A1 – monotonie 1M paires
- adversarial tests A2 – fuzzing seuil
- adversarial tests B1 – collision Merkle 100K
- adversarial tests C1 – seal tamper
- adversarial tests D1 – consensus split fail-closed
- adversarial tests E1 – signature tamper
- smoke tests
- runtime probes
- OpenAPI snapshot
- MANIFEST_SHA256.json

- README_FREEZE.md
- audit reports
- replay reports
- decision traces
- hash reports
- freeze manifests
- rfc3161_anchor.json
- merkle_root.json
- metadata.json

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d’action contrôlés
- Zone risques et limites

### Actions autorisées

- Lancer test
- Lire output
- Comparer freeze
- Créer demande humaine
- Marquer ENV_BLOCKED

### Actions interdites

- Déclarer proof PASS sans run
- ignorer fail
- modifier output

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-10-process-validation-center/summary
- GET /api/workcenter/4-10-process-validation-center/items
- GET /api/workcenter/4-10-process-validation-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.

- Test cohérence statut vérité.

### Critères d'acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.10-001 Créer page.
- FE-4.10-002 Créer composants principaux.
- BE-4.10-001 Créer endpoint summary.
- BE-4.10-002 Créer endpoint details.
- QA-4.10-001 Tests états et permissions.

## 4.11 Proof & Replay Lab

### Objectif de la page

Cette page doit rendre accessibles les preuves et le replay de décisions. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Proof & Replay Lab doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- Lean
- 57 théorèmes / lemmes
- lake build
- TLA
- TLC
- compute_merkle_root.py
- compute_merkle_standalone.py
- verify_all.py
- verify_decision.py
- verify_merkle.py
- Merkle
- RFC3161
- PROOFKIT_REPORT.json
- proofs/examples/README.md
- proofs/examples/manual_merkle_annotated.md
- proofs/examples/merkle_root_2201.md
- adversarial tests A1 – monotonie 1M paires

- adversarial tests A2 – fuzzing seuil
- adversarial tests B1 – collision Merkle 100K
- adversarial tests C1 – seal tamper
- adversarial tests D1 – consensus split fail-closed
- adversarial tests E1 – signature tamper
- smoke tests
- runtime probes
- OpenAPI snapshot
- MANIFEST_SHA256.json
- README_FREEZE.md
- audit reports
- replay reports
- decision traces
- hash reports
- freeze manifests
- rfc3161_anchor.json
- merkle_root.json
- metadata.json

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- lake build
- verify_all
- verify_decision
- verify_merkle
- replay decision
- compare manifests

### Actions interdites

- modifier proof
- rejouer avec mutation
- publier proof incomplet

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-11-proof-replay-lab/summary
- GET /api/workcenter/4-11-proof-replay-lab/items
- GET /api/workcenter/4-11-proof-replay-lab/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef

- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.11-001 Créer page.
- FE-4.11-002 Créer composants principaux.
- BE-4.11-001 Créer endpoint summary.
- BE-4.11-002 Créer endpoint details.
- QA-4.11-001 Tests états et permissions.

## 4.12 Resource Library

### Objectif de la page
Cette page doit indexer tous les fichiers, guides, proofs, connecteurs, schemas, freezes, audits. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Resource Library doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- connectors/bank_normal_flow.py
- sigma/domains/bank_agents.py
- connectors/trading_live.py
- sigma/domains/trading_agents.py
- sigma/examples/trading_domain_example_06.py
- connectors/aviation_robo.py
- sigma/domains/gps_defense_aviation_agents.py

- sigma/domains/ecom_agents.py
- connectors/context_packet_flow.py
- connectors/interface_ready_memory.py
- connectors/memory_feedback_candidate.py
- connectors/_decision_card_fr.py
- sigma/domains/meta_agents.py
- connectors/brody_memory_readonly_adapter.py
- NCP readonly stub – à conserver comme stub non souverain tant que le connecteur n'est pas implémenté
- Scraping readonly stub – aucun réseau externe, aucun fetch, aucun crawl
- Path Compute V0 advisory only – pas de décision, pas d'action, pas de mutation
- sigma/aggregation.py
- sigma/base.py
- sigma/connectors.py
- sigma/contracts.py
- sigma/evaluate.py
- sigma/graphiti_readonly_bridge.py
- sigma/guard.py
- sigma/obsidia_sigma_v130.py
- sigma/orchestrator_preview.py
- sigma/packets.py
- sigma/protocols.py
- sigma/registry.py
- sigma/run_pipeline.py
- sigma/sigma_config.json
- sigma/sigma_monitor.py
- sigma/trees_activation_readonly.py
- sigma/domains/bank_agents.py
- sigma/domains/ecom_agents.py
- sigma/domains/gps_defense_aviation_agents.py
- sigma/domains/meta_agents.py
- sigma/domains/trading_agents.py
- sigma/stress_test_results.json
- Lean
- 57 théorèmes / lemmes
- lake build
- TLA
- TLC
- compute_merkle_root.py
- compute_merkle_standalone.py
- verify_all.py
- verify_decision.py
- verify_merkle.py
- Merkle
- RFC3161
- PROOFKIT_REPORT.json
- proofs/examples/README.md
- proofs/examples/manual_merkle_annotated.md
- proofs/examples/merkle_root_2201.md
- adversarial tests A1 – monotonie 1M paires

- adversarial tests A2 – fuzzing seuil
- adversarial tests B1 – collision Merkle 100K
- adversarial tests C1 – seal tamper
- adversarial tests D1 – consensus split fail-closed
- adversarial tests E1 – signature tamper
- smoke tests
- runtime probes
- OpenAPI snapshot
- MANIFEST_SHA256.json
- README_FREEZE.md
- audit reports
- replay reports
- decision traces
- hash reports
- freeze manifests
- rfc3161_anchor.json
- merkle_root.json
- metadata.json

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Chercher
- Filtrer
- Lire
- Lier à module
- Marquer legacy
- Créer candidate

### Actions interdites

- Supprimer source
- remplacer vérité source
- mélanger legacy et actif

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-12-resource-library/summary
- GET /api/workcenter/4-12-resource-library/items
- GET /api/workcenter/4-12-resource-library/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef

- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.12-001 Créer page.
- FE-4.12-002 Créer composants principaux.
- BE-4.12-001 Créer endpoint summary.
- BE-4.12-002 Créer endpoint details.
- QA-4.12-001 Tests états et permissions.

## 4.13 Session Ledger

### Objectif de la page
Cette page doit reconstruire toute session de travail, run, proof, mémoire ou décision. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Session Ledger doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- session_id
- branch_id
- trace_id
- span_id
- domain
- agent
- connectors_invoked

- sigma_run_id
- decisions
- proof_refs
- ledger_hash
- status
- files_touched
- risks

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Ouvrir timeline
- Exporter ledger
- Créer freeze
- Comparer sessions
- Créer recap

### Actions interdites

- modifier ledger clos
- supprimer événement
- changer décision

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-13-session-ledger/summary
- GET /api/workcenter/4-13-session-ledger/items
- GET /api/workcenter/4-13-session-ledger/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.13-001 Créer page.
- FE-4.13-002 Créer composants principaux.
- BE-4.13-001 Créer endpoint summary.
- BE-4.13-002 Créer endpoint details.
- QA-4.13-001 Tests états et permissions.

## 4.14 Freeze Factory

### Objectif de la page
Cette page doit préparer et tracer les freezes sans les truquer. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Freeze Factory doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- LOCAL_FREEZE
- RUNTIME_FREEZE
- PROOF_FREEZE
- BRODY_FREEZE
- GRAPHITI_FREEZE
- DOMAIN_FREEZE
- SESSION_FREEZE
- WORKCENTER_FREEZE
- MANIFEST_SHA256.json
- README_FREEZE.md
- runtime_probe.txt
- test_output.txt
- openapi_snapshot.json
- hash_report.json
- registry_snapshot.json

### Widgets et composants UI
- Titre pédagogique
- Bandeau statut
- Tableau filtrable

- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d’action contrôlés
- Zone risques et limites

### Actions autorisées
- Créer freeze draft
- Calculer manifest
- Smoke
- Demander validation
- Comparer
- Publier après validation

### Actions interdites
- modifier freeze
- publier sans manifest
- omettre fichier
- freeze actif sans smoke

### Routes backend à appeler ou à créer
- GET /api/workcenter/4-14-freeze-factory/summary
- GET /api/workcenter/4-14-freeze-factory/items
- GET /api/workcenter/4-14-freeze-factory/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés
- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.14-001 Créer page.
- FE-4.14-002 Créer composants principaux.

- BE-4.14-001 Créer endpoint summary.
- BE-4.14-002 Créer endpoint details.
- QA-4.14-001 Tests états et permissions.

## 4.15 Coding Sandbox

### Objectif de la page

Cette page doit permettre la lecture, simulation, test et proposition de patch sans casser le système. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Coding Sandbox doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- READ_ONLY_LEARNING
- TEST_ONLY
- PATCH_PROPOSAL
- LOCAL_SANDBOX_WRITE
- HUMAN_APPROVED_WRITE
- KERNEL_TOUCH_FORBIDDEN
- safe_to_read
- safe_to_test
- safe_to_patch
- requires_review
- kernel_protected
- secrets_forbidden
- legacy_unknown

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Lire fichier
- Chercher symbole
- Simuler connecteur
- Générer diff

- Créer patch proposal
- Run test safe

### Actions interdites

- éditer kernel
- éditer env
- delete freeze
- rewrite proofs
- commit direct
- merge direct
- write graphiti
- emit act

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-15-coding-sandbox/summary
- GET /api/workcenter/4-15-coding-sandbox/items
- GET /api/workcenter/4-15-coding-sandbox/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.15-001 Créer page.
- FE-4.15-002 Créer composants principaux.
- BE-4.15-001 Créer endpoint summary.
- BE-4.15-002 Créer endpoint details.
- QA-4.15-001 Tests états et permissions.

## 4.16 Server Control Advisory

### Objectif de la page
Cette page doit surveiller et proposer des commandes serveur sans exécution directe. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Server Control Advisory doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- API 8000
- Kernel 3001
- Graphiti 8011
- Brody 8012
- UI 5173
- Neo4j 7474
- Neo4j 7687
- logs
- PID
- uptime
- CPU
- mémoire

### Widgets et composants UI
- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées
- Lire status
- Copier commande
- Créer validation request
- Créer runtime freeze

### Actions interdites
- start direct
- stop direct
- kill process direct

- show secrets

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-16-server-control-advisory/summary
- GET /api/workcenter/4-16-server-control-advisory/items
- GET /api/workcenter/4-16-server-control-advisory/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.16-001 Créer page.
- FE-4.16-002 Créer composants principaux.
- BE-4.16-001 Créer endpoint summary.
- BE-4.16-002 Créer endpoint details.
- QA-4.16-001 Tests états et permissions.

## 4.17 Multimodal Workbench

### Objectif de la page

Cette page doit importer et classer images, audio, vidéo, PDF, captures, logs, schémas, slides, terminal. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Multimodal Workbench doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit

raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- image
- audio
- vidéo
- PDF
- captures écran
- logs
- schémas
- slides
- terminal
- transcriptions
- annotations
- hash

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Importer
- Annoter
- Créer resource
- Créer memory candidate
- Lier module
- Créer freeze multimodal

### Actions interdites

- décider depuis image
- écrire mémoire canon
- exécuter code du fichier
- ignorer secret

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-17-multimodal-workbench/summary
- GET /api/workcenter/4-17-multimodal-workbench/items
- GET /api/workcenter/4-17-multimodal-workbench/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag

- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d'acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.17-001 Créer page.
- FE-4.17-002 Créer composants principaux.
- BE-4.17-001 Créer endpoint summary.
- BE-4.17-002 Créer endpoint details.
- QA-4.17-001 Tests états et permissions.

## 4.18 Audit Center

### Objectif de la page
Cette page doit centraliser les audits, risques, do-not-claim, statuts de vérité et limites. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur
Pour un non-développeur, Audit Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher
- audits
- risques
- claims interdits
- statuts vérité
- preuves manquantes
- legacy
- documented_only
- env_blocked

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d’action contrôlés
- Zone risques et limites

### Actions autorisées

- Lire audit
- Créer risk
- Lier proof
- Exporter rapport
- Marquer limite

### Actions interdites

- masquer limite
- sur-claim
- confondre prior audit et runtime

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-18-audit-center/summary
- GET /api/workcenter/4-18-audit-center/items
- GET /api/workcenter/4-18-audit-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.18-001 Créer page.
- FE-4.18-002 Créer composants principaux.
- BE-4.18-001 Créer endpoint summary.
- BE-4.18-002 Créer endpoint details.
- QA-4.18-001 Tests états et permissions.

## 4.19 Governance Policies Center

### Objectif de la page

Cette page doit gérer les règles, seuils, permissions, modes et exceptions. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l'état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible

Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Governance Policies Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- policy_id
- scope
- threshold
- mode
- permission
- exception
- approver
- date
- proof_required

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Créer policy draft
- Demander validation
- Comparer policy
- Exporter

### Actions interdites
- activer policy sans validation
- override kernel
- bypass BLOCK

### Routes backend à appeler ou à créer
- GET /api/workcenter/4-19-governance-policies-center/summary
- GET /api/workcenter/4-19-governance-policies-center/items
- GET /api/workcenter/4-19-governance-policies-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés
- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail
- ActionRequest

### Tests à prévoir
- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation
- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum
- FE-4.19-001 Créer page.
- FE-4.19-002 Créer composants principaux.
- BE-4.19-001 Créer endpoint summary.
- BE-4.19-002 Créer endpoint details.
- QA-4.19-001 Tests états et permissions.

## 4.20 Backlog & Delivery Center

### Objectif de la page
Cette page doit convertir ce cahier des charges en tickets et suivre la réalisation. Elle transforme une zone technique complexe en surface lisible, vérifiable et gouvernable. Elle ne doit pas être décorative : chaque donnée affichée doit aider à comprendre l’état réel du projet ou à déclencher une validation contrôlée.

### Utilisateur cible
Utilisateur non développeur, développeur, auditeur et copilote IA. La page doit commencer par une explication simple, puis donner accès aux détails techniques pour ceux qui en ont besoin.

### Explication non-développeur

Pour un non-développeur, Backlog & Delivery Center doit être présenté comme une salle spécialisée du cockpit. On n'y vient pas pour tout faire, mais pour comprendre un pan précis du système. La page doit raconter ce qu'elle montre, pourquoi ces données comptent, quelles actions sont sûres, quelles actions sont interdites et quelle preuve permet de faire confiance à l'information affichée.

### Données à afficher

- epic
- ticket
- priorité
- dépendance
- effort
- statut
- acceptance criteria
- tests
- owner
- freeze cible

### Widgets et composants UI

- Titre pédagogique
- Bandeau statut
- Tableau filtrable
- Panneau détail
- Timeline ou graphe si pertinent
- JSON viewer pour données brutes
- Boutons d'action contrôlés
- Zone risques et limites

### Actions autorisées

- Créer ticket
- Prioriser
- Filtrer
- Exporter backlog
- Lier freeze

### Actions interdites

- clôturer sans test
- supprimer acceptances
- mélanger done et prepared

### Routes backend à appeler ou à créer

- GET /api/workcenter/4-20-backlog-delivery-center/summary
- GET /api/workcenter/4-20-backlog-delivery-center/items
- GET /api/workcenter/4-20-backlog-delivery-center/details/:id
- POST /api/workcenter/validation/request si action de validation nécessaire

### Modèles de données concernés

- WorkcenterItem
- StatusBadge
- EvidenceRef
- RiskFlag
- AuditTrail

- ActionRequest

### Tests à prévoir

- Test état vide.
- Test erreur API.
- Test source legacy.
- Test action interdite.
- Test export.
- Test cohérence statut vérité.

### Critères d’acceptation

- La page explique son rôle sans jargon.
- Toutes les données clés sont affichées.
- Les actions interdites sont bloquées.
- Les détails techniques restent accessibles.
- Un utilisateur peut comprendre quoi faire ensuite.

### Tickets dev minimum

- FE-4.20-001 Créer page.
- FE-4.20-002 Créer composants principaux.
- BE-4.20-001 Créer endpoint summary.
- BE-4.20-002 Créer endpoint details.
- QA-4.20-001 Tests états et permissions.

# Partie 5 — Modèles de données détaillés

## ServiceStatus

Le modèle ServiceStatus doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- id
- name
- port
- role
- status
- truth_level
- last_probe_at
- last_freeze_id
- pid
- uptime
- cpu_percent
- memory_mb
- logs_ref
- risk_flags

## InvariantStatus

Le modèle InvariantStatus doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- decision_authority
- api_role
- allowed_to_decide
- readonly
- advisory_only
- emits_act
- emits_verdict
- memory_write
- graphiti_write
- kernel_mutation
- neo4j_write
- mcp_bridge
- path_compute
- real_action
- canonical_write

## DomainSpec

Le modèle DomainSpec doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- domain_id
- label
- status
- payload_schema

- example_payloads
- connector
- adapter
- api_route
- kernel_route
- risk_metrics
- last_runtime
- last_freeze
- last_replay
- last_decision
- allowed_modes
- forbidden_modes

## ConnectorSpec

Le modèle ConnectorSpec doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- connector_id
- path
- domain
- role
- read_only
- stub
- contract_ref
- tests_ref
- last_run
- last_error
- risk_level

## MemoryEntry

Le modèle MemoryEntry doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- memory_id
- memory_type
- status
- source
- session_id
- created_at
- updated_at
- ttl
- size
- hash
- validation_status
- freeze_ref
- risk_flags

## ValidationRequest

Le modèle ValidationRequest doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- validation_id
- type
- target_type
- target_id
- reason
- risk_level
- requested_by
- assigned_to
- created_at
- due_at
- status
- decision
- comment
- evidence_refs

## ProofRun

Le modèle ProofRun doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- proof_id
- proof_type
- script
- status
- started_at
- ended_at
- duration
- output_ref
- manifest_ref
- freeze_ref
- error_message

## SessionLedgerEntry

Le modèle SessionLedgerEntry doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- session_id
- branch_id
- trace_id
- span_id
- memory_type
- status
- start_time
- end_time
- connectors_invoked
- sigma_run_id

- brody_modules
- decisions
- proof_refs
- ledger_hash
- files_touched
- risks
- next_commands

## FreezeRecord

Le modèle FreezeRecord doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- freeze_id
- freeze_type
- scope
- created_at
- created_by
- manifest_sha256_ref
- readme_ref
- runtime_probe_ref
- test_output_ref
- openapi_snapshot_ref
- hash_report_ref
- registry_snapshot_ref
- status
- validation_ref

## KnowledgeItem

Le modèle KnowledgeItem doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- knowledge_id
- source_type
- source_ref
- domain
- classification
- status
- summary
- conflicts
- validation_ref
- canon_ref
- resource_ref
- memory_ref

## AuditEvent

Le modèle AuditEvent doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- event_id
- event_type

- timestamp
- project_id
- domain_id
- agent_id
- session_id
- trace_id
- span_id
- repo
- branch
- worktree
- risk_level
- knowledge_status
- payload
- cic_context

### BacklogTicket

Le modèle BacklogTicket doit être défini côté backend et documenté pour le frontend. Il permet d'éviter que l'interface construise ses propres règles. Tous les champs doivent être sérialisables en JSON et documentés dans l'OpenAPI du cockpit.

- ticket_id
- epic
- title
- description
- priority
- dependencies
- effort
- status
- acceptance_criteria
- tests
- owner
- freeze_target

## Partie 6 — API Contract du cockpit

Les routes suivantes sont à créer pour le cockpit. Elles ne remplacent pas les routes existantes Obsidia. Elles agrègent, documentent et exposent des états de lecture. Les routes POST autorisées créent des demandes de validation ou de freeze ; elles ne réalisent pas l'action finale sans validation.

- GET /api/workcenter/status
- GET /api/workcenter/metrics
- GET /api/workcenter/registries
- GET /api/workcenter/components
- GET /api/workcenter/services
- GET /api/workcenter/routes
- GET /api/workcenter/invariants
- GET /api/workcenter/authority/graph
- GET /api/workcenter/runtime/summary
- GET /api/workcenter/runtime/services
- GET /api/workcenter/runtime/logs/:service_id
- GET /api/workcenter/domains
- GET /api/workcenter/domains/:domain_id
- GET /api/workcenter/connectors
- GET /api/workcenter/connectors/:connector_id
- GET /api/workcenter/sigma/summary
- GET /api/workcenter/sigma/modules
- GET /api/workcenter/sigma/runs
- GET /api/workcenter/sigma/runs/:run_id
- GET /api/workcenter/memory/summary
- GET /api/workcenter/memory/entries
- GET /api/workcenter/memory/entries/:memory_id
- POST /api/workcenter/memory/candidate
- GET /api/workcenter/knowledge/items
- GET /api/workcenter/knowledge/items/:knowledge_id
- POST /api/workcenter/knowledge/candidate
- GET /api/workcenter/brody/modules
- GET /api/workcenter/brody/modules/:module_id
- POST /api/workcenter/brody/exercise-result
- GET /api/workcenter/validation/human
- GET /api/workcenter/validation/process
- POST /api/workcenter/validation/request
- POST /api/workcenter/validation/:validation_id/approve
- POST /api/workcenter/validation/:validation_id/reject
- GET /api/workcenter/proofs
- GET /api/workcenter/proofs/:proof_id
- POST /api/workcenter/proofs/run-request
- GET /api/workcenter/replay/decisions
- GET /api/workcenter/replay/decisions/:decision_id
- POST /api/workcenter/replay/request
- GET /api/workcenter/resources
- GET /api/workcenter/resources/:resource_id
- POST /api/workcenter/resources/import-request
- GET /api/workcenter/sessions
- GET /api/workcenter/sessions/:session_id

- POST /api/workcenter/session/close/dry-run
- GET /api/workcenter/freezes
- GET /api/workcenter/freezes/:freeze_id
- POST /api/workcenter/freezes/request
- GET /api/workcenter/sandbox/files
- GET /api/workcenter/sandbox/file
- POST /api/workcenter/sandbox/patch-proposal
- POST /api/workcenter/sandbox/test-request
- GET /api/workcenter/server/advisory
- POST /api/workcenter/server/advisory/request
- GET /api/workcenter/multimodal/items
- POST /api/workcenter/multimodal/import-request
- GET /api/workcenter/audit/events
- GET /api/workcenter/audit/events/:event_id
- GET /api/workcenter/governance/policies
- POST /api/workcenter/governance/policy-draft
- GET /api/workcenter/backlog/epics
- GET /api/workcenter/backlog/tickets
- POST /api/workcenter/backlog/ticket

## Règles générales API

- Chaque réponse contient un champ status.
- Chaque réponse contient un champ truth_level : CONFIRMED_BY_RUNTIME, CONFIRMED_BY_TEST, CONFIRMED_BY_FREEZE, CONFIRMED_BY_CODE, PRESENT_NOT_CONNECTED, DOCUMENTED_ONLY, LEGACY, BLOCKED, FAILED, ENV_BLOCKED, UNKNOWN.
- Chaque objet critique contient evidence_refs.
- Chaque route POST crée une demande ou un draft sauf si l'action finale a déjà été validée.
- Aucune route cockpit n'appelle directement /kernel/ragnarok sans passer par les routes existantes et les règles de gouvernance.
- Aucune route cockpit n'écrit dans Graphiti en V1.

# Partie 7 — Workflows complets

## Workflow A — Lire l'état global

1. L'utilisateur ouvre Dashboard global.
2. Le frontend appelle GET /api/workcenter/status et GET /api/workcenter/metrics.
3. Le backend agrège les services, invariants, proofs, mémoires, validations et freezes.
4. L'UI affiche les statuts réels et les alertes.
5. Si une couche est fermée ou frozen, l'UI explique pourquoi et propose une action advisory.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow B — Ajouter un savoir

6. L'utilisateur importe un document, une note, un PDF, une capture ou un log.
7. Le Multimodal Workbench crée une Resource.
8. Le Knowledge Center classe la ressource par domaine et type.
9. Une Memory Candidate est créée avec source, hash, résumé et risk flags.
10. La candidate passe en NEEDS_HUMAN_REVIEW ou NEEDS_PROCESS_REVIEW.
11. Après validation, elle peut devenir Training Memory, Domain Memory, Proof Memory, Operational Memory ou Canonical Memory.
12. Un freeze optionnel scelle l'état validé.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow C — Valider une mémoire

13. Une Memory Candidate apparaît dans Memory Governance Center.
14. L'utilisateur ouvre son détail et lit sources, risques, conflits, evidence_refs.
15. Il crée une validation request.
16. Human Validation Center assigne un validateur.
17. Le validateur approuve, rejette, demande test, demande freeze ou downgrade vers scratch.
18. Le statut mémoire change et l'événement est inscrit au Session Ledger.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow D — Exécuter une preuve

19. L'utilisateur ouvre Proof & Replay Lab.
20. Il choisit verify_all.py, verify_decision.py, verify_merkle.py, lake build, TLA/TLC ou un adversarial test.
21. Le backend crée une Process Validation Request.
22. Le Proof Runner exécute dans un environnement isolé.
23. Le résultat est stocké avec logs, output, statut et evidence_refs.
24. Si PASS, un proof freeze peut être demandé.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow E — Rejouer une décision

25. L'utilisateur sélectionne une décision par decision_id ou session_id.
26. Le Replay Lab charge payload, votes, aggregate, kernel verdict, CIC receipt, proof refs et freeze refs.
27. L'UI affiche la timeline et la raison du verdict.
28. Si le replay diffère du résultat original, l'UI crée une alerte sécurité.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow F — Ajouter un domaine

29. L'utilisateur ouvre Domain Center.
30. Il crée un draft domaine avec nom, description, connecteur prévu, payload schema, modes autorisés, modes interdits.
31. Le domaine est DOCUMENTED_ONLY.
32. Un connecteur est ajouté en sandbox ou patch proposal.
33. Des tests sont écrits.
34. Sigma run en SIMULATED.
35. Process validation.
36. Freeze domaine.
37. Activation READONLY_LIVE uniquement après validation.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow G — Créer un freeze

38. L'utilisateur sélectionne un scope : runtime, proof, brody, graphiti, domain, session, workcenter.
39. Freeze Factory prépare un draft.
40. Le backend collecte README_FREEZE.md, MANIFEST_SHA256.json, git_status.txt, test_output.txt, runtime_probe.txt, openapi_snapshot.json, hash_report.json, registry_snapshot.json.
41. Un smoke test vérifie la cohérence.
42. Une validation humaine ou process est demandée.
43. Le freeze est publié si validé.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

## Workflow H — Proposer un patch

44. L'utilisateur ouvre Coding Sandbox.
45. Il lit les fichiers en safe_to_read.
46. Il génère un patch proposal.
47. Le système vérifie zones protégées, secrets et kernel_protected.
48. Le patch devient une Memory Candidate ou Patch Proposal.
49. Human Validation Center examine.
50. Si approuvé, tests puis freeze.
51. Le merge réel se fait hors du cockpit ou dans une phase validée.

Critère de réussite : chaque étape produit une trace. Si une étape échoue, le workflow s'arrête en état explicite et ne passe pas en mode permissif. Le cockpit doit afficher l'erreur, l'état, le responsable et l'action suivante possible.

# Partie 8 — Sécurité by design détaillée

## Autorité unique

Le cockpit doit rappeler partout que seul Kernel X108 décide. FastAPI est BRIDGE_ONLY, Brody est éducatif, Graphiti est readonly, Sigma agrège et CIC trace. Cette séparation d'autorité est une règle d'architecture et une règle de sécurité.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Readonly by default

Toutes les surfaces ouvertes en V1 sont readonly ou advisory. Le cockpit peut créer une demande, pas réaliser l'action finale. Une écriture de mémoire, un freeze publié, une policy active ou un patch appliqué passent par validation.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Fail-closed

L'ordre BLOCK > HOLD > ALLOW doit être présenté comme un invariant fondamental. Une ambiguïté, une contradiction, un timeout ou une absence de kernel ne donne jamais un ALLOW par défaut.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Guard every ingress

Toutes les entrées utilisateur, fichiers, documents multimodaux, patchs, payloads et prompts passent par des guards. Brody utilise Two-Level Guard, forbidden action keys, safe sentinel values et renvoie PACK_INJECT_BLOCKED ou P54_SANDBOX_NO_ACT si nécessaire.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Causal traceability

Chaque action critique doit être reliée à trace_id, span_id, session_id, CIC context et éventuellement receipt. Le cockpit ne doit jamais afficher une décision orpheline sans contexte.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Tamper evidence

Les freezes, manifests SHA256, Merkle roots, RFC3161 anchors et proof reports forment la preuve d'intégrité. Le cockpit doit rendre ces éléments lisibles.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Protected scope

server.kernel.sealed.cjs, contracts.broken-ragnarok.py, fichiers secrets, env, proofs scellées et freezes publiés sont zones protégées. Le sandbox ne peut pas les modifier.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Claims discipline

Ne jamais afficher production-ready, certifié, sécurité absolue, Graphiti décide, Brody décide, cockpit décide, mémoire autorise, domaine autorise seul. Utiliser les statuts de vérité.

Exigence dev : la règle doit être encodée dans le backend, affichée dans le frontend, testée en QA et documentée dans l'audit center. Si la règle n'est qu'un texte de documentation, elle est insuffisante.

## Do not claim — liste complète

- production-ready
- certifié
- sécurité absolue
- Brody décide
- Graphiti décide
- la mémoire autorise
- les domaines autorisent seuls
- l'UI peut muter le kernel
- 57 théorèmes comme preuve totale du système complet sans préciser le périmètre
- Graphiti runtime si seulement freeze
- Kernel runtime si seulement freeze
- TLA fully checked si TLC non relancé dans l'audit courant
- RFC3161 fully verified si openssl verify non exécuté
- adversarial tests current repo si seulement prior audit

# Partie 9 — Backlog dev détaillé

## EPIC 0 — Baseline et sources

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 0-T01 — Inventorier toutes les sources Obsidia/X108 du chat et fichiers fournis

Description : Inventorier toutes les sources Obsidia/X108 du chat et fichiers fournis. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 0-T02 — Créer source_inventory.md

Description : Créer source_inventory.md. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 0-T03 — Créer baseline runtime freeze

Description : Créer baseline runtime freeze. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 0-T04 — Lister les ports et statuts

Description : Lister les ports et statuts. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 0-T05 — Lister les invariants

Description : Lister les invariants. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 0-T06 — Lister les modules confirmés et non confirmés

Description : Lister les modules confirmés et non confirmés. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 1 — Workcenter backend foundation

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 1-T01 — Créer routes/workcenter.py

Description : Créer routes/workcenter.py. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 1-T02 — Créer modèles Pydantic

Description : Créer modèles Pydantic. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 1-T03 — Créer registries JSON

Description : Créer registries JSON. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 1-T04 — Créer endpoint /status

Description : Créer endpoint /status. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 1-T05 — Créer endpoint /metrics

Description : Créer endpoint /metrics. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 1-T06 — Créer endpoint /registries

Description : Créer endpoint /registries. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 1-T07 — Créer tests unitaires

Description : Créer tests unitaires. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 2 — Frontend shell

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 2-T01 — Créer layout cockpit

Description : Créer layout cockpit. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 2-T02 — Créer navigation principale

Description : Créer navigation principale. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 2-T03 — Créer design system badges

Description : Créer design system badges. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 2-T04 — Créer service API client

Description : Créer service API client. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 2-T05 — Créer état loading/error/empty

Description : Créer état loading/error/empty. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 2-T06 — Créer moteur de recherche global

Description : Créer moteur de recherche global. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 3 — Dashboard et Authority

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 3-T01 — Créer Dashboard global

Description : Créer Dashboard global. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 3-T02 — Créer Authority Graph

Description : Créer Authority Graph. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 3-T03 — Créer composants cartes

Description : Créer composants cartes. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 3-T04 — Créer bandeau invariants

Description : Créer bandeau invariants. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 3-T05 — Créer export status report

Description : Créer export status report. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 3-T06 — Créer tests UI

Description : Créer tests UI. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 4 — Runtime et Server Advisory

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 4-T01 — Créer runtime center

Description : Créer runtime center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 4-T02 — Créer service probes

Description : Créer service probes. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 4-T03 — Créer log viewer

Description : Créer log viewer. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 4-T04 — Créer server advisory requests

Description : Créer server advisory requests. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 4-T05 — Créer alertes port

Description : Créer alertes port. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 4-T06 — Créer runtime freeze draft

Description : Créer runtime freeze draft. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 5 — Domain et Sigma

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 5-T01 — Créer Domain Center

Description : Créer Domain Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 5-T02 — Créer fiches domaines

Description : Créer fiches domaines. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 5-T03 — Créer connectors registry

Description : Créer connectors registry. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 5-T04 — Créer Sigma Center

Description : Créer Sigma Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 5-T05 — Créer pipeline viewer

Description : Créer pipeline viewer. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 5-T06 — Créer simulation run_pipeline dry-run

Description : Créer simulation run_pipeline dry-run. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 6 — Memory et Knowledge

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 6-T01 — Créer Memory Governance Center

Description : Créer Memory Governance Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 6-T02 — Créer memory registry

Description : Créer memory registry. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 6-T03 — Créer memory candidate creation

Description : Créer memory candidate creation. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 6-T04 — Créer Knowledge Center

Description : Créer Knowledge Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 6-T05 — Créer classification statuses

Description : Créer classification statuses. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 6-T06 — Créer validation linking

Description : Créer validation linking. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 7 — Brody Education

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 7-T01 — Créer modules T1 à T20

Description : Créer modules T1 à T20. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 7-T02 — Créer quiz engine

Description : Créer quiz engine. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 7-T03 — Créer exercise runner

Description : Créer exercise runner. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 7-T04 — Créer progression tracking

Description : Créer progression tracking. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 7-T05 — Créer guard visibility

Description : Créer guard visibility. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 7-T06 — Créer storage résultats

Description : Créer storage résultats. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 8 — Validation

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 8-T01 — Créer Human Validation Center

Description : Créer Human Validation Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 8-T02 — Créer Process Validation Center

Description : Créer Process Validation Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 8-T03 — Créer validation requests

Description : Créer validation requests. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 8-T04 — Créer approve/reject workflows

Description : Créer approve/reject workflows. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 8-T05 — Créer SLA views

Description : Créer SLA views. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 8-T06 — Créer audit trail

Description : Créer audit trail. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 9 — Proof & Replay

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 9-T01 — Créer Proof Lab

Description : Créer Proof Lab. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 9-T02 — Intégrer verify_all

Description : Intégrer verify_all. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 9-T03 — Intégrer verify_decision

Description : Intégrer verify_decision. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 9-T04 — Intégrer verify_merkle

Description : Intégrer verify_merkle. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 9-T05 — Intégrer lake build

Description : Intégrer lake build. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 9-T06 — Créer replay viewer

Description : Créer replay viewer. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 9-T07 — Créer proof status matrix

Description : Créer proof status matrix. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 10 — Resource Library

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 10-T01 — Créer resources registry

Description : Créer resources registry. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 10-T02 — Indexer README

Description : Indexer README. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 10-T03 — Indexer audits

Description : Indexer audits. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 10-T04 — Indexer proofs

Description : Indexer proofs. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 10-T05 — Indexer connectors

Description : Indexer connectors. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 10-T06 — Créer search filters

Description : Créer search filters. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 10-T07 — Créer preview viewer

Description : Créer preview viewer. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 11 — Session Ledger

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 11-T01 — Créer sessions registry

Description : Créer sessions registry. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 11-T02 — Créer timeline

Description : Créer timeline. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 11-T03 — Créer export JSON

Description : Créer export JSON. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 11-T04 — Créer session close dry-run

Description : Créer session close dry-run. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 11-T05 — Créer ledger freeze link

Description : Créer ledger freeze link. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 11-T06 — Créer compare sessions

Description : Créer compare sessions. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 12 — Freeze Factory

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 12-T01 — Créer freeze request wizard

Description : Créer freeze request wizard. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 12-T02 — Créer manifest generator

Description : Créer manifest generator. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 12-T03 — Créer README_FREEZE generator

Description : Créer README_FREEZE generator. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 12-T04 — Créer smoke check

Description : Créer smoke check. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 12-T05 — Créer freeze compare

Description : Créer freeze compare. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

#### EPIC 12-T06 — Créer publish after validation

Description : Créer publish after validation. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 13 — Coding Sandbox

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

#### EPIC 13-T01 — Créer file browser

Description : Créer file browser. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

#### EPIC 13-T02 — Créer code viewer

Description : Créer code viewer. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

#### EPIC 13-T03 — Créer symbol search

Description : Créer symbol search. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 13-T04 — Créer patch proposal

Description : Créer patch proposal. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 13-T05 — Créer test request

Description : Créer test request. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 13-T06 — Créer protected zones enforcement

Description : Créer protected zones enforcement. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 14 — Multimodal

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 14-T01 — Créer upload

Description : Créer upload. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 14-T02 — Créer media viewer

Description : Créer media viewer. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 14-T03 — Créer annotations

Description : Créer annotations. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 14-T04 — Créer transcription placeholder

Description : Créer transcription placeholder. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 14-T05 — Créer resource creation

Description : Créer resource creation. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 14-T06 — Créer multimodal freeze request

Description : Créer multimodal freeze request. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

## EPIC 15 — Audit et Governance

Objectif : transformer le cahier des charges en livrables vérifiables. Chaque ticket doit avoir un statut PREPARED, IN_PROGRESS, BLOCKED, DONE ou FROZEN. Aucun ticket ne passe à DONE sans test ou justification explicite.

### EPIC 15-T01 — Créer Audit Center

Description : Créer Audit Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 15-T02 — Créer risk registry

Description : Créer risk registry. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 15-T03 — Créer do-not-claim page

Description : Créer do-not-claim page. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 15-T04 — Créer Governance Policies Center

Description : Créer Governance Policies Center. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 15-T05 — Créer policy drafts

Description : Créer policy drafts. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

### EPIC 15-T06 — Créer policy compare

Description : Créer policy compare. Le développeur doit créer ou modifier les fichiers nécessaires sans toucher au kernel scellé. Le ticket doit produire un résultat visible dans le cockpit ou un endpoint backend exploitable.

Critères d'acceptation : code livré, test exécuté ou raison ENV_BLOCKED documentée, aucun invariant violé, trace inscrite dans le Session Ledger, ticket lié à un freeze si le changement est structurant.

Tests minimum : test état nominal, test état vide, test erreur, test permission, test non-régression. Si le ticket touche aux preuves, ajouter test proof. Si le ticket touche aux mémoires, ajouter test memory governance. Si le ticket touche aux serveurs, ajouter test readonly.

# Partie 10 — Planning de build

## MVP 0 — Cahier et baseline

Produire le présent cahier des charges, inventorier les sources, créer le freeze baseline, valider le scope.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

## MVP 1 — Backend cockpit readonly

Créer les registries, les routes GET de base, les agrégations status/metrics, tests unitaires.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

## MVP 2 — Frontend cockpit skeleton

Créer l'application, navigation, dashboard, authority graph, runtime center.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

## MVP 3 — Domain, Sigma, Memory

Créer Domain Center, Sigma Center, Memory Governance Center et Knowledge Center.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

## MVP 4 — Validation et Brody

Créer Human Validation, Process Validation et Brody Education T1 à T20.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

## MVP 5 — Proof, Replay, Resource

Créer Proof Lab, Replay Lab, Resource Library et intégrer les proofs.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

## MVP 6 — Ledger et Freeze

Créer Session Ledger et Freeze Factory avec manifests et smoke.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.

- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

### MVP 7 — Sandbox, Server, Multimodal

Créer Coding Sandbox, Server Advisory et Multimodal Workbench.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

### V1 — Cockpit complet readonly

Toutes les pages fonctionnelles en lecture, validations par demandes, aucun write direct.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

### V1.5 — Workbench contrôlé

Ajout d'un workbench type Claude Code limité : terminal, diff, patch proposal, sandbox, validation.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

### V2 — Actions validées

Permettre certaines actions après validation humaine/process, toujours sans bypass kernel.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

### V3 — Audit externe

Préparer documentation, preuves, replays et freezes pour audit indépendant.

- Livrable : fichier ou module fonctionnel.
- Validation : tests et revue humaine.
- Freeze : obligatoire si la phase modifie architecture, sécurité, mémoire ou preuve.
- Risque : ne pas passer à la phase suivante si les invariants ne sont pas visibles.

# Partie 11 — Annexes exhaustives

## Annexe A — Modules cockpit

- Dashboard global
- Authority Graph
- Runtime Center
- Domain Center
- Sigma Center
- Memory Governance Center
- Knowledge Center
- Brody Education Center
- Human Validation Center
- Process Validation Center
- Proof & Replay Lab
- Resource Library
- Session Ledger
- Freeze Factory
- Coding Sandbox
- Server Control Advisory
- Multimodal Workbench
- Audit Center
- Governance Policies Center
- Backlog & Delivery Center

## Annexe B — Domaines

- Bank
- Trading
- GPS
- Aviation
- Defense
- Ecom
- CIC
- NCP
- Scraping readonly
- Meta
- futurs domaines identifiés dans les sources : ecommerce, defence, space, biology, healthcare

## Annexe C — Connecteurs

- connectors/bank_normal_flow.py
- sigma/domains/bank_agents.py
- connectors/trading_live.py
- sigma/domains/trading_agents.py
- sigma/examples/trading_domain_example_06.py
- connectors/aviation_robo.py
- sigma/domains/gps_defense_aviation_agents.py
- sigma/domains/ecom_agents.py
- connectors/context_packet_flow.py
- connectors/interface_ready_memory.py
- connectors/memory_feedback_candidate.py
- connectors/_decision_card_fr.py

- sigma/domains/meta_agents.py
- connectors/brody_memory_readonly_adapter.py
- NCP readonly stub – à conserver comme stub non souverain tant que le connecteur n'est pas implémenté
- Scraping readonly stub – aucun réseau externe, aucun fetch, aucun crawl
- Path Compute V0 advisory only – pas de décision, pas d'action, pas de mutation

## Annexe D — Modules Sigma

- sigma/aggregation.py
- sigma/base.py
- sigma/connectors.py
- sigma/contracts.py
- sigma/evaluate.py
- sigma/graphiti_readonly_bridge.py
- sigma/guard.py
- sigma/obsidia_sigma_v130.py
- sigma/orchestrator_preview.py
- sigma/packets.py
- sigma/protocols.py
- sigma/registry.py
- sigma/run_pipeline.py
- sigma/sigma_config.json
- sigma/sigma_monitor.py
- sigma/trees_activation_readonly.py
- sigma/domains/bank_agents.py
- sigma/domains/ecom_agents.py
- sigma/domains/gps_defense_aviation_agents.py
- sigma/domains/meta_agents.py
- sigma/domains/trading_agents.py
- sigma/stress_test_results.json

## Annexe E — Proof stack

- Lean
- 57 théorèmes / lemmes
- lake build
- TLA
- TLC
- compute_merkle_root.py
- compute_merkle_standalone.py
- verify_all.py
- verify_decision.py
- verify_merkle.py
- Merkle
- RFC3161
- PROOFKIT_REPORT.json
- proofs/examples/README.md
- proofs/examples/manual_merkle_annotated.md
- proofs/examples/merkle_root_2201.md
- adversarial tests A1 – monotonie 1M paires
- adversarial tests A2 – fuzzing seuil
- adversarial tests B1 – collision Merkle 100K

- adversarial tests C1 – seal tamper
- adversarial tests D1 – consensus split fail-closed
- adversarial tests E1 – signature tamper
- smoke tests
- runtime probes
- OpenAPI snapshot
- MANIFEST_SHA256.json
- README_FREEZE.md
- audit reports
- replay reports
- decision traces
- hash reports
- freeze manifests
- rfc3161_anchor.json
- merkle_root.json
- metadata.json

## Annexe F — Clés CIC

- authority
- memory_write
- memory_type
- decision_authority
- api_role
- emit_act
- env
- epoch
- session_id
- ledger_hash
- pii_masking
- audit_mode
- freeze_after
- prover
- sigma_run_id
- brody_t
- terrain_branch
- proof_required
- user_intent
- component
- gate

## Annexe G — Types et statuts de mémoire

- SessionScratch
- Candidate
- Frozen
- KX108Ongoing
- Sigma
- BridgeCache
- Human Validated Memory
- Process Validated Memory
- Training Memory
- Operational Memory

- Domain Memory
- Proof Memory
- Canonical Memory
- Rejected Memory
- Deprecated Memory
- Forbidden Secret
- RAW_CAPTURE
- SESSION_SCRATCH
- CANDIDATE
- NEEDS_HUMAN_REVIEW
- NEEDS_PROCESS_REVIEW
- PROCESS_VALIDATED
- HUMAN_VALIDATED
- PROMOTED_TO_CANON
- PROMOTED_TO_TRAINING
- PROMOTED_TO_OPERATIONAL
- PROMOTED_TO_DOMAIN
- PROMOTED_TO_PROOF
- REJECTED
- SUPERSEDED
- DEPRECATED
- FORBIDDEN_SECRET
- ADVISORY
- PENDING_FREEZE
- FROZEN
- ARCHIVED

## Annexe H — Modules Brody Education T1 à T20

T1 — Comprendre Obsidia / X108 : Découvrir le projet, la séparation des rôles et le Kernel X108.

T2 — Autorité et non-souveraineté : Comprendre pourquoi seuls certains composants peuvent décider et pourquoi les autres doivent rester readonly.

T3 — Kernel X108 : Comprendre le noyau, le endpoint /kernel/ragnarok, ALLOW, HOLD et BLOCK.

T4 — API FastAPI readonly : Lire le rôle BRIDGE_ONLY, les 164 routes, les flags de non-action et l'état runtime.

T5 — Bank domain : Lire les flux bancaires, les décisions ALLOW S1 et le connecteur bank_normal_flow.py.

T6 — Trading domain : Lire les flux trading, les décisions BLOCK S4 et la raison CONTRADICTION_THRESHOLD_REACHED.

T7 — GPS / Aviation / Defense domain : Lire les signaux GPS, aviation, défense et la posture critique readonly.

T8 — Brody education layer : Comprendre le rôle de Brody comme module éducatif non souverain.

T9 — Graphiti readonly memory : Comprendre Graphiti V20, 167 nœuds, 477 relations, 20 épisodes, mode FROZEN_READONLY.

T10 — CIC / causal context : Comprendre les 21 clés du contexte causal et les receipts.

T11 — Proof stack Lean / TLA / Python : Comprendre Lean, TLA, Merkle, RFC3161 et les scripts verify.

T12 — Freezes / manifests / audit reports : Lire MANIFEST_SHA256.json, README_FREEZE.md, PROOFKIT_REPORT.json.

T13 — Server launch and healthchecks : Comprendre les ports 3001, 8000, 8011, 8012, 5173, 7474, 7687.

T14 — Debugging without breaking kernel : Savoir diagnostiquer sans toucher au kernel scellé.

T15 — Patch proposal workflow : Proposer un patch sans l'appliquer directement, avec diff, validation et freeze.

T16 — Coding sandbox : Lire, tester et simuler sans modifier les zones protégées.

T17 — Replay / decision envelope : Rejouer une décision, lire decision_id, trace_id, attestation_ref, evidence_refs.

T18 — Market / pitch / non-tech explanation : Expliquer Obsidia sans sur-claim et sans langage trop technique.

T19 — Security boundaries : Comprendre les invariants de sécurité et les flags false.

T20 — Operational deployment readiness : Lister ce qui manque avant une exploitation sensible ou publique.

## Annexe I — Statuts de vérité

- CONFIRMED_BY_RUNTIME
- CONFIRMED_BY_TEST
- CONFIRMED_BY_FREEZE
- CONFIRMED_BY_CODE
- CONFIRMED_BY_PRIOR_AUDIT
- PRESENT_NOT_CONNECTED
- PRESENT_NOT_RUN
- PRESENT_BUT_NOT_EXECUTED
- DOCUMENTED_ONLY
- LEGACY
- BLOCKED
- FAILED
- ENV_BLOCKED
- UNKNOWN
- NON CONFIRMÉ DANS LES SOURCES FOURNIES
- LISTE PARTIELLE — SOURCE INCOMPLÈTE

## Annexe J — Modes domaines

- SIMULATED
- RECORDED
- REPLAY
- READONLY_LIVE
- DRY_RUN_ACTION
- HUMAN_APPROVED_ACTION
- KERNEL_GOVERNED_ACTION
- REAL_ACTION_BLOCKED
- REAL_ACTION_WITHOUT_X108 — interdit
- DIRECT_ACT — interdit
- DOMAIN_DECIDES_ALONE — interdit
- UNVERIFIED_LIVE — interdit

## Annexe K — Zones Coding Sandbox

- READ_ONLY_LEARNING
- TEST_ONLY

- PATCH_PROPOSAL
- LOCAL_SANDBOX_WRITE
- HUMAN_APPROVED_WRITE
- KERNEL_TOUCH_FORBIDDEN
- safe_to_read
- safe_to_test
- safe_to_patch
- requires_review
- kernel_protected
- secrets_forbidden
- legacy_unknown

## Annexe L — Livrables attendus par sprint

- Code source frontend
- Code source backend
- Modèles Pydantic ou JSON Schema
- Routes OpenAPI documentées
- Tests unitaires
- Tests intégration
- Tests permissions
- Tests readonly
- Test état vide
- Test erreur API
- Test invariant authority
- README de module
- Registry mise à jour
- Session Ledger entry
- Freeze si changement structurant

# Partie 12 — Guide de lecture pour un non-développeur

## Kernel X108

Quand un développeur ou un auditeur parle de Kernel X108, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. le noyau souverain. Il est le seul composant autorisé à produire un verdict final ALLOW, HOLD ou BLOCK.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## ALLOW

Quand un développeur ou un auditeur parle de ALLOW, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. l'action candidate est autorisée par le kernel.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## HOLD

Quand un développeur ou un auditeur parle de HOLD, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. l'action est suspendue. Elle demande plus de preuve, plus de temps ou une validation humaine.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## BLOCK

Quand un développeur ou un auditeur parle de BLOCK, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. l'action est interdite. C'est l'état le plus restrictif.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Fail-closed

Quand un développeur ou un auditeur parle de Fail-closed, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. si un doute, une contradiction ou une panne existe, le système choisit HOLD ou BLOCK plutôt qu'un ALLOW permissif.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## FastAPI Bridge

Quand un développeur ou un auditeur parle de FastAPI Bridge, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. l'API qui expose des routes au cockpit. Elle transmet et lit, mais elle ne décide pas.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Sigma

Quand un développeur ou un auditeur parle de Sigma, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. la couche qui agrège les votes des agents de domaine et prépare un paquet pour le kernel.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Brody

Quand un développeur ou un auditeur parle de Brody, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. la couche d'éducation et d'assistance. Elle explique, apprend et propose, mais ne décide pas.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Graphiti

Quand un développeur ou un auditeur parle de Graphiti, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. la mémoire graphe en mode readonly, utilisée pour naviguer dans la connaissance déjà indexée.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## CIC

Quand un développeur ou un auditeur parle de CIC, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. Causal Identity Context. C'est l'enveloppe causale d'un appel, avec 21 clés fixes pour prouver le rôle non souverain du composant.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Proof Lab

Quand un développeur ou un auditeur parle de Proof Lab, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. l'espace qui exécute ou consulte les preuves Lean, TLA, Merkle, RFC3161 et les scripts de vérification.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Freeze

Quand un développeur ou un auditeur parle de Freeze, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve affichable. un état figé, hashé et documenté. Une fois freeze, il sert de preuve d'état.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Session Ledger

Quand un développeur ou un auditeur parle de Session Ledger, il ne faut pas le lire comme un mot abstrait. Dans le cockpit, ce concept doit correspondre à une page, une fiche, un statut ou une preuve

affichable. le journal d'une session : ce qui s'est passé, quel agent a agi, quelle décision a été prise, quelles preuves existent.

Exigence cockpit : le concept doit être visible quelque part dans l'interface. Si un concept central n'a aucune page, aucun badge, aucun détail et aucune trace, alors le cockpit ne permet pas de piloter correctement le projet.

Exemple utilisateur : un utilisateur non technique doit pouvoir cliquer sur ce concept, lire une explication courte, voir les données réelles associées, comprendre le statut et savoir quelle action est possible ou interdite.

## Conclusion opérationnelle

Ce cahier des charges définit un cockpit complet, narratif et exploitable. La priorité n'est pas de créer une interface brillante mais creuse. La priorité est de construire une tour de contrôle qui respecte Obsidia : lecture avant action, preuve avant claim, freeze avant merge, validation avant canonisation, kernel avant décision, audit avant déploiement.

Le développeur qui reçoit ce document doit commencer par les registries et les routes backend readonly, puis construire les pages une par une. L'ordre recommandé est : baseline, backend cockpit, frontend shell, dashboard, authority graph, runtime, domain, sigma, memory, validation, Brody, proof, resources, ledger, freeze, sandbox, server, multimodal, audit, governance, backlog. Chaque module doit avoir son état vide, son état erreur, ses tests, ses critères d'acceptation et son lien avec les invariants.

Ce document doit être traité comme une base V1. Il peut être étendu, mais il ne doit pas être compressé. Tout ajout futur doit conserver la même règle : pas de raccourci, pas de liste incomplète cachée, pas de sur-claim et pas d'autorité inventée.